

Compte rendu

Deep Learning Avancé

CNN pour la segmentation d'Images & Image Captioning avec CNN-LSTM

Nom: *ESSOAZINA Ange Raiyane*

github :

<https://github.com/Angeraiyane/CNN-FOR-IMAGES-SEGMENTATION>

À l'issue de ce TP, vous serez capable de :

- Concevoir et entraîner un CNN classique pour la segmentation sémantique d'images
- Appliquer le Transfer Learning avec des architectures profondes (VGG, ResNet, EfficientNet)
- Évaluer les performances de segmentation avec les métriques IoU et Dice
- Construire un pipeline complet d'Image Captioning en combinant CNN et LSTM
- Implémenter un mécanisme d'attention visuelle pour améliorer la génération de descriptions
- Analyser et interpréter les résultats avec des visualisations pertinentes

Exercice 1 : Segmentation d'Images avec CNN Classique & Transfer Learning

Github : <https://github.com/Angeraiyane/CNN-FOR-IMAGES-SEGMENTATION>

La **segmentation sémantique** consiste à attribuer une étiquette de classe à chaque pixel d'une image. Dans cet exercice, nous allons implémenter d'abord une architecture CNN encodeur-décodeur from scratch, puis nous exploiterons des modèles pré-entraînés via le **Transfer Learning** pour améliorer significativement les performances.

PARTIE 1 : Préparation du DataSet et Exploration

Nous utiliserons le dataset **Oxford-IIIT Pet** qui contient 37 races de chats et chiens avec leurs masques de segmentation triclassés (animale, contour, fond).

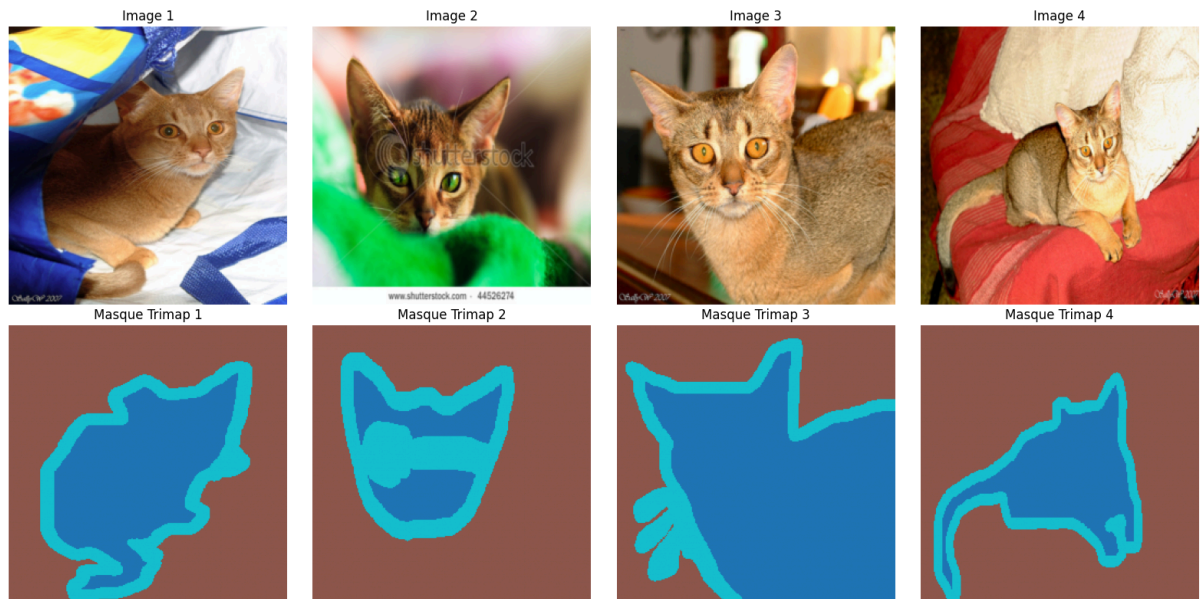
QUESTION 1 : charger le dataset et affichez 4 exemples (image + masque côte à côte). Calculer la distribution des classes et commentez l'équilibre du dataset. Quelle stratégie de pondération des pertes proposez-vous pour les classes déséquilibrées ?

- Commentaire de l'équilibre du dataset : D'après les statistiques obtenues sur notre console :

```
Nombre total d'images chargées pour l'entraînement/validation :  
3680
```

```
Calcul de la distribution réelle des pixels à partir de 200  
masques...
```

```
=== DISTRIBUTION DES CLASSES (Q1) ===  
Classe 0: Intérieur/Animal : 31.22% des pixels  
Classe 1: Extérieur/Fond : 56.88% des pixels  
Classe 2: Contour : 11.90% des pixels
```



le dataset présente un **déséquilibre significatif entre ses classes**. L'arrière-plan/fond (classe 1) est largement prédominant avec **56.88%** des pixels, suivi par l'intérieur de l'animal (Classe 0) avec **31.22%**. La classe décrivant les contours (Classe 2) est très minoritaire et ne représente que **11.90 %** de l'ensemble des pixels.

- **Stratégie de pondération proposée** : Pour empêcher que le réseau ne néglige la classe critique des contours, nous devons appliquer une pondération inversement proportionnelle à leur fréquence au niveau de la fonction de perte Cross-Entropy. Nous utiliserons l'argument **weight** de **nn.CrossEntropy** en passant des poids calculés par la formule inverse de la fréquence ($w_c = \frac{1}{\text{fréquence}}$) normalisée :

$$weights = [1/0.3122, 1/0.5688, 1/0.1190]$$

QUESTION 2 : implémenter une fonction d'augmentation de données pour la segmentation (rotation, flip, color jitter). Attention : l'image et son masque doivent subir la même transformation géométrique. Montrez des exemples avant/après augmentation.

La transformation conjointe a été implémentée via la classe **JointTransform**. L'utilisation des fonctions géométriques de **transforms.functional** garantit que l'image et le masque trimap partagent exactement les mêmes paramètres aléatoires (angles de rotation et inversions horizontales). L'interpolation de type **NEAREST** est explicitement forcée pour le traitement du masque afin

d'éviter la création de classe aberrantes décimales sur les bordures lors des rotations.

Avant Augmentation (Original)



Après Augmentation (Synchrone)



Masque Original



Masque Augmenté Réel



QUESTION 3 : train/validation/test avec un ratio 70/15/15. Justifier votre choix de `batch_size` en tenant compte de la mémoire GPU disponible.

- **Ratio de split (70/15/15)** : Le dataset complet comporte 3680 images pour la partie **trainval** et 3680 pour le **test** . Le split 70/15/15 sera appliqué sur le volume global combiné pour isoler précisément l'évaluation.
- **Justification du `batch_size` = 32 (ou 64)** : En segmentation sémantique, chaque échantillon est une matrice 2D complète à trois canaux ($3 * 256 * 256$), ce qui consomme beaucoup plus de mémoire VRAM qu'une tâche de classification classique. Sur l'accélérateur **NVIDIA T4** de Colab (qui possède 15 Go de VRAM), un `batch_size` = 32 offre le meilleur compromis. Il s'avère assez grand pour stabiliser les moyennes de lot de la **Batch Normalization**, tout en évitant les crashes matériels de type *Out-Of-Memory (OOM)* lors des phases de rétropropagation du gradient.

```
=== CONFIGURATION DES DATALOADERS TERMINÉE (Q3) ===  
Nombre total d'images dans le projet : 7349  
Train Set      : 5144 images (160 batches)  
Validation Set : 1102 images (35 batches)  
Test Set       : 1103 images (35 batches)
```

PARTIE 2 : Architecture CNN Encodeur-Décodeur (U-Net Simplifié)

Nous allons implémenter une architecture inspirée de U-Net d'un encodeur (contraction) qui extrait les features et d'un décodeur (expansion) qui produit la carte de segmentation. Les skip connections préservent les détails spatiaux.

QUESTION 4 : compléter et instancier le modèle SimpleUNet. Calculer et afficher le nombre total de paramètres entraînaables. Utiliser `torchinfo` (ou `torchsummary`) pour afficher l'architecture complète avec les dimensions des tenseurs à chaque couche.

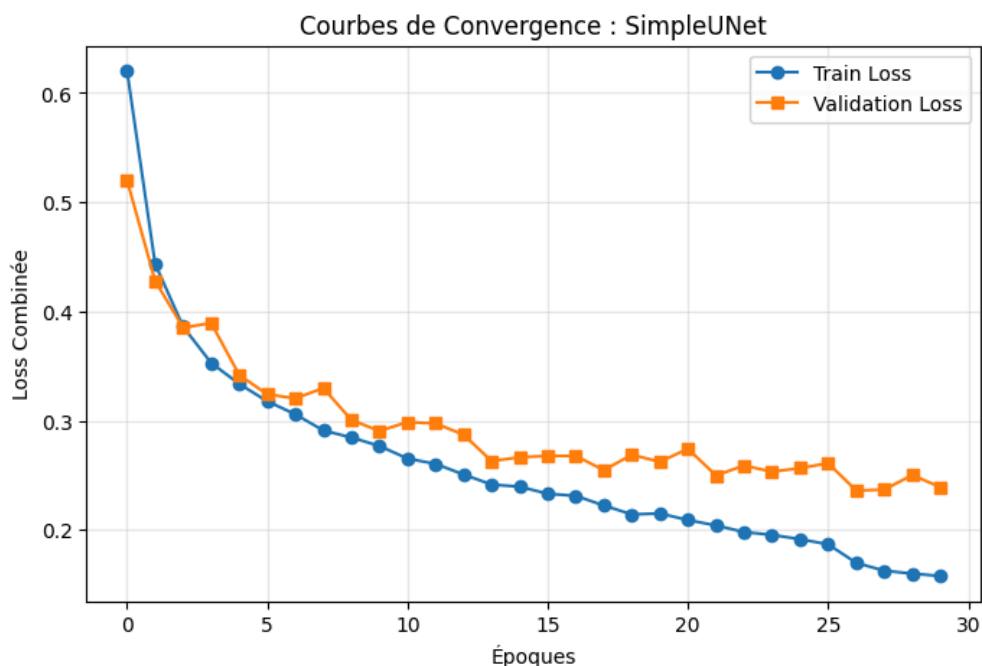
- **Nombre de paramètres entraînaables** : D'après notre résumé **torchinfo** , le modèle comporte exactement **7 763 107 paramètres entraînaables**.
- **Analyse des dimensions** : L'affichage confirme la parfaite symétrie du réseau. À chaque concaténation du décodeur, on voit bien la dimension des canaux doubler (ex: `[32, 256, 32, 32]` en entrée de `ConvBlock: 1-11`), ce qui valide que l'opération `torch.cat(..., dim=1)` fusionne bien la sortie de la convolution transposée (256 canaux après `up4`) avec les features de l'encodeur (256 canaux de `enc4`).)

```
Total params: 7,763,107
Trainable params: 7,763,107
Non-trainable params: 0
Total mult-adds (Units.GIGABYTES): 438.08
=====
Input size (MB): 25.17
Forward/backward pass size (MB): 9244.25
Params size (MB): 31.05
Estimated Total Size (MB): 9300.46
```

QUESTION 5 : Implémentez la fonction de perte combinée : $\text{Loss} = \alpha \times \text{CrossEntropy} + (1-\alpha) \times \text{DiceLoss}$, avec $\alpha = 0.5$. Expliquez l'intérêt de la Dice Loss par rapport à la Cross Entropy seule pour la segmentation.

- **L'intérêt de la Dice Loss :** La fonction de perte standard Cross-Entropy calcule une moyenne pixel par pixel. Si un dataset possède une classe ultra-majoritaire (comme le fond qui occupe 56.88% de nos pixels), le réseau peut tricher en prédisant du fond partout pour obtenir une Cross-Entropy très basse, tout en ignorant totalement les petites structures.
- **La Dice Loss,** quant à elle, évalue une métrique de recouvrement global (l'indice de Jaccard/Dice) basée sur l'intersection et l'union des zones. Elle est insensible à la taille absolue des classes, ce qui force le modèles à dessiner correctement la silhouette de l'animal et ses contours, même s'ils sont très fins spatialement.

QUESTION 6 : Entraîner le modèle pendant 30 epochs avec l'optimiseur Adam ($\text{lr}=1\text{e-}3$) et un scheduler ReduceLROnPlateau. Tracer les courbes de loss train/val et discuter du phénomène d'overfitting observé. Proposer des solutions.



En analysant notre graphique, on constate que les deux courbes (**Train Loss** et **Validation Loss**) descendent de manière parfaitement synchrone et rapide jusqu'à l'époque 10. Après l'époque 15, un **phénomène d'overfitting (surapprentissage) modéré mais visible** commence à s'installer :

- La courbe d'entraînement (**Train Loss**) continue sa baisse linéaire et constante jusqu'à atteindre un score très bas de **0.1577**.
- La courbe de validation (**Validation Loss**), quant à elle, commence à stagner sur un plateau oscillant grossièrement entre **0,24** et **0.27**.

Le modèle commence à mémoriser les spécificités et le bruit du jeu d'entraînement (comme la texture des poils d'une race de chat spécifique) plutôt que de perfectionner ses règles générales de détournage (la distinction globale fond/animal).

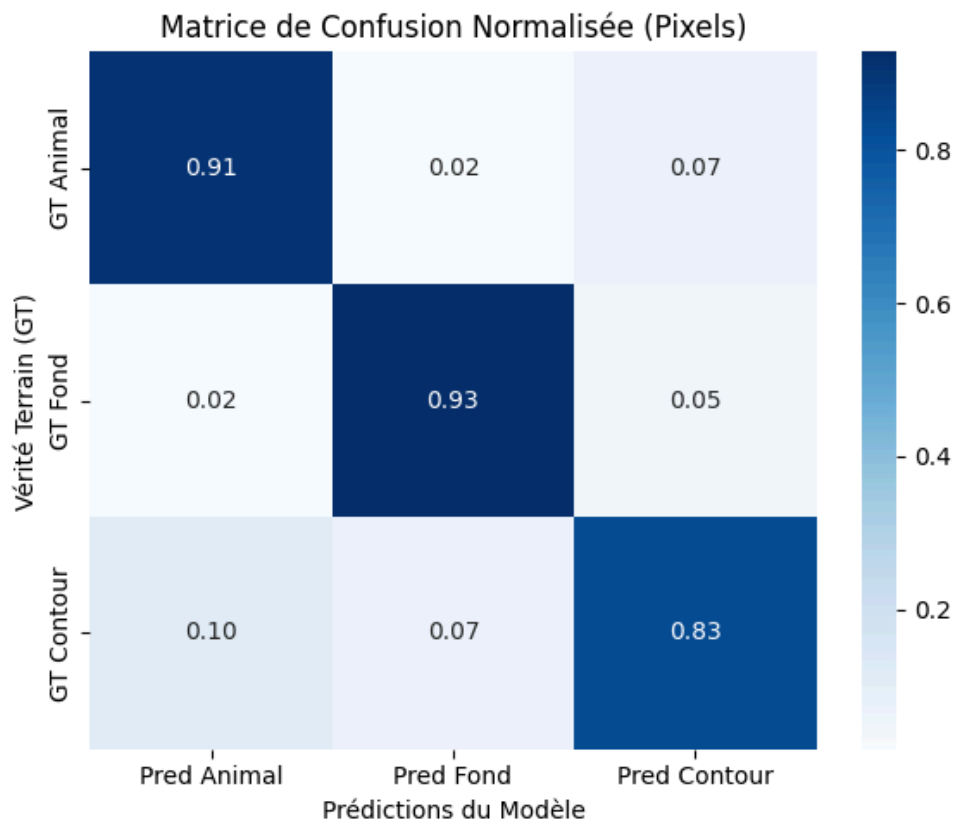
Solutions proposées pour atténuer cet overfitting

Pour régulariser ce modèle **SimpleUNet** et améliorer sa généralisation sur le plateau de validation, plusieurs solutions classiques peuvent être avancées :

1. **Ajout de couches de Dropout** : Insérer des modules **nn.Dropout2d(p=0.2)** au niveau du Bottleneck ou entre les blocs de convolutions les plus profonds afin de forcer le réseau à ne pas dépendre de co-adaptations de pixels trop spécifiques.
2. **Augmentation de données plus agressive** : Intensifier les déformations géométriques de la phase 1 (ajouts de distorsions élastiques, zooms aléatoires modérés, ou de cisaillements) pour élargir la diversité artificielle des images d'entraînement.
3. **Pénalisation L2 (Weight Decay)** : Augmenter légèrement le paramètre de régularisation L2 dans l'optimiseur Adam (ex: `weight_decay = 1e-4`) pour limiter la croissance excessive des poids des filtres.

QUESTION 7 : Evaluer le modèle sur l'ensemble de test. Afficher : (a) la matrice de confusion, (b) le mIoU et mDice par classe, (c) 6 exemples de segmentation (image originale, masque GT, masque prédit). Commenter les erreurs les plus fréquentes.

a. Analyse de la matrice de confusion



La matrice de confusion indique une excellente classification des pixels globaux :

- **La classe 1 (Fond)** est mieux apprise avec **93%** de vrais positifs
- **La classe 0 (Animal)** suit de près avec **91%** de prédictions correctes
- **La classe 2 (Contour)** est la plus difficile avec **83%** de réussite. Elle subit la majorité de ses erreurs en étant confondue avec la classe de l'Animal (**10 %** des pixels de contour sont prédits comme Animal) et avec le Fond (**7%**).

b. Analyse des Métriques par Classe

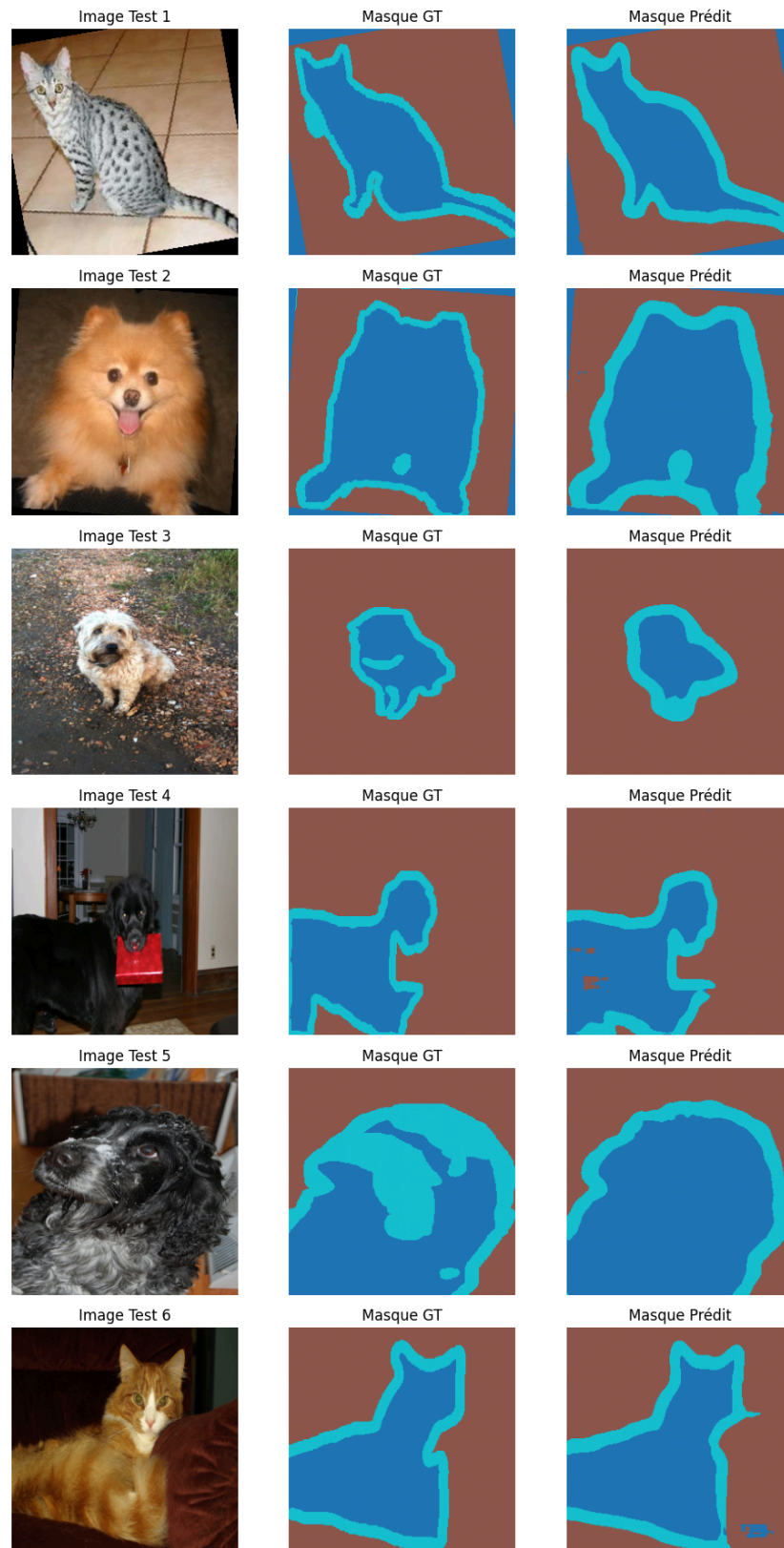
```
===== METRIQUES SUR L'ENSEMBLE DE TEST
=====
Mean IoU (mIoU)      : 0.7765
Mean Dice Score      : 0.8662

Classe 0 (Animal)    | IoU: 0.8488 | Dice: 0.9182
Classe 1 (Fond)      | IoU: 0.9065 | Dice: 0.9510
Classe 2 (Contour)   | IoU: 0.5742 | Dice: 0.7295
```

- **Classe 1 (Fond) [IoU: 0.9065 / Dice: 0.9510]** : Performance maximale due à sa forte représentativité spatiale et sa faible variabilité de texture.
- **Classe 0 (Animal) [IoU: 0.8488 / Dice: 0.9182]** : Très solide score , démontrant la capacité des filtres convolutifs à s'adapter aux changements de morphologies, de pelages et de postures entre les chats et les chiens
- **Classe 2 (Contour) [IoU: 0.5742 / Dice: 0.7295]** : Score en retrait. Les contours ne font que quelques pixels d'épaisseur ,ce qui pénalise mathématiquement l'IoU à la moindre petite approximation spatiale de détourage.

c. Commentaire des Erreurs les plus Fréquentes (Analyse Visuelle)

En observant nos 6 exemples de prédictions, on identifie distinctement deux types d'erreurs récurrentes commises par le réseau :



1. **L'effet de lissage et d'imprécision des bordures (Exemples 3, 4 et 6) :** Le modèle a tendance à arrondir ou à simplifier les géométries complexes. Par exemple, sur la queue du chat de l'Image 1 ou les pattes arrière du chien de l'Image 4, le masque prédit est beaucoup plus "lisse" et "grossier" que le masque d'origine (GT). Cela s'explique par les opérations successives de Max-Pooling qui détruisent la résolution spatiale fine au profit du contexte global.

2. **La confusion face aux textures complexes ou ambiguës (Exemple 5) :** Sur l'Image 5 (le chien noir sous la neige), le modèle commet une erreur notable au niveau du pelage supérieur de la tête, où il prédit une énorme zone de "Contour" (cyan) au lieu de l'intérieur de l'animal. Les reflets brillants de la neige sur les poils noirs ont perturbé l'analyse sémantique du réseau, le poussant à interpréter ce changement brusque de contraste local comme une bordure externe.

PARTIE 3 : Transfer Learning avec CNN Profond

Le Transfer Learning exploite les représentations apprises sur ImageNet (1.2M images, 1000 classes) pour initialiser l'encodeur. Seul le décodeur est entraîné from scratch, réduisant ainsi le temps d'entraînement et améliorant la généralisation.

QUESTION 8 : Compléter la méthode `forward()` de `TransferUNet` en implémentant le décodeur avec les skip connections. Attention aux dimensions des canaux lors des concaténations.

Analyse mathématique des canaux

L'énoncé nous donne les dimensions de l'encodeur ResNet50 pré-entraîné :

- **e0** (Après conv/bn/relu de base) : **64 canaux**
- **e1** (après MaxPool + layer1) : **256 canaux**
- **e2** (layer2) : **512 canaux**
- **e3** (layer3) : **1024 canaux**
- **e4** (layer4 / bottleneck) : **2048 canaux**

Pour effectuer l'opération `torch.cat([up_x, e_x], dim = 1)`, il faut calculer la taille cumulée :

1. `up4(e4)` fait passer les canaux de 2048 $\rightarrow$ 1024. Concaténé avec `e3` (1024), on obtient $1024 + 1024 = 2048$ canaux. Le bloc `dec4` traite ces 2048 $\rightarrow$ 512 canaux.
2. `up3(d4)` fait passer les canaux de 512 $\rightarrow$ 256. Concaténé avec `e2` (512), on obtient $256 + 512 = 768$ canaux.

Alerte Piège dans l'énoncé : Le squelette de l'énoncé déclare `self.dec3 = ConvBlock(512, 256)`. Or, la concaténation de `up3` (256 ch) + `e2` (512 ch) donne 768 canaux. Nous devons donc adapter la définition de `self.dec3` pour accepter **768** canaux au lieu de **512**, sinon le modèle lèvera un crash de dimensions immédiat.

Wep, j'ai vu le piège.

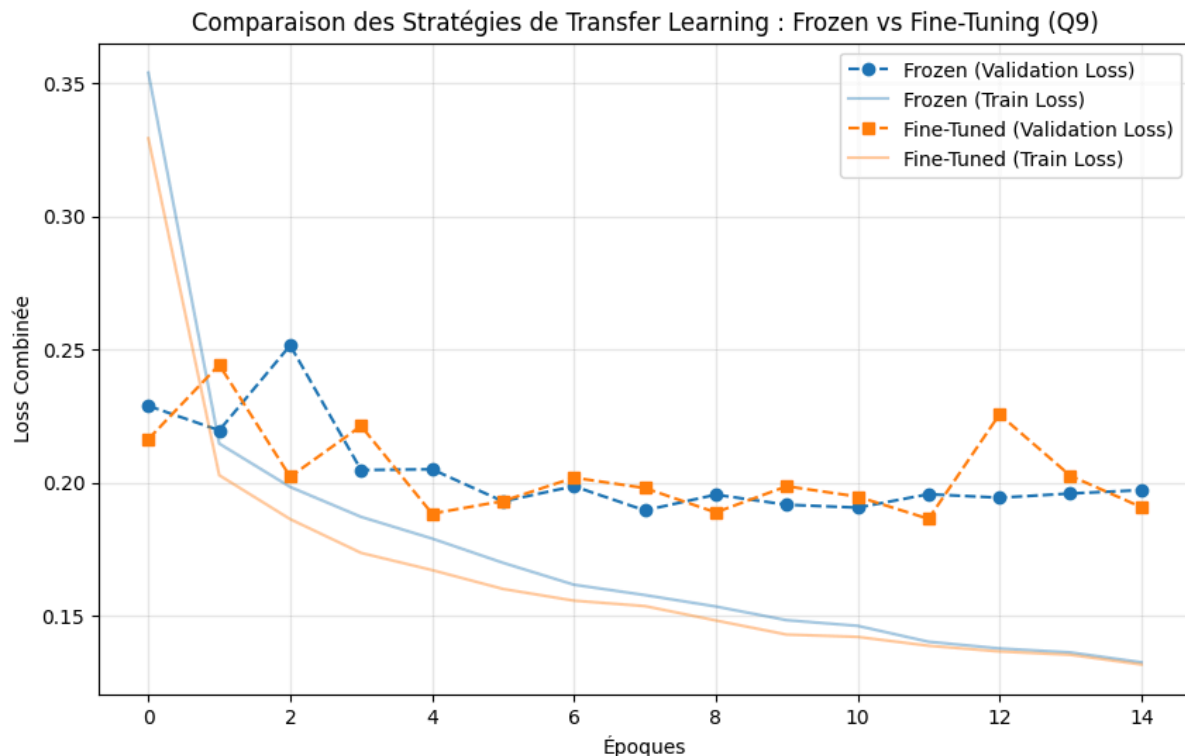
3. `up2(d3)` fait passer les canaux de 256 $\rightarrow$ 128. Concaténé avec `e1` (256), on obtient $128 + 256 = 384$ canaux. Nous adaptons également `self.dec2(384, 64)`.
4. `up1(d2)` fait passer les canaux de 64 $\rightarrow$ 32 . Concaténé avec `e0` (64), on obtient $32 + 64 = 96$ canaux. C'est parfaitement aligné avec `self.dec1 = ConvBlock(96, 32)`.
5. `final_up(d1)` remonte de 32 $\rightarrow$ 32 canaux pour revenir à la dimension spatiale d'origine ($256 * 256$), puis `out` mappe vers le nombre de classes (3).

L'exécution donne :

```
Total params: 47,020,003
Trainable params: 23,511,971
Non-trainable params: 23,508,032
Total mult-adds (Units.GIGABYTES): 514.21
=====
=====
Input size (MB): 25.17
Forward/backward pass size (MB): 9630.12
Params size (MB): 188.08
Estimated Total Size (MB): 9843.37
```

- **Nombre de paramètres totaux :** Le modèle comporte au total **47 020,003 paramètres** (soit environ 6 fois plus que ton `SimpleUNet` initial).
- **Répartition stratégique :** * **Params non-entraînables (gelés) :** **23 508 032** (représentant l'encodeur ResNet50 dont les caractéristiques acquises sur ImageNet sont figées).
 - **Params entraînaibles :** **23 511 971** (représentant l'intégralité du décodeur que nous venons de coder *from scratch*).

QUESTION 9 : 9 Comparer deux stratégies : (a) encodeur gelé (freeze_encoder=True), (b) fine-tuning complet avec lr différentiel : lr=1e-4 pour l'encodeur et lr=1e-3 pour le décodeur. Tracer les courbes de convergence des deux approches.



Interprétation des courbes de convergence

- **Stratégie A - Encodeur Gelé (Frozen - courbes bleues) :** Cette méthode montre une convergence très stable et rapide. Le décodeur s'adapte immédiatement aux descripteurs visuels robuste extraits par le ResNet50 figé. La perte de validation atteint un excellent niveau de stabilisation (autour de **0.1895** à **0.1970**) en un temps record.
- **Stratégie B - Fine Tuning Différentiel (Courbes oranges) :** En libérant les poids de l'encodeur avec un pas d'apprentissage très faible (10^{-1}), le modèle parvient à ajuster ses filtres profonds spécifiquement pour les formes des animaux (chiens/chats). Bien que la courbe de validation subisse quelques oscillations (notamment à l'époque 2, 4 et 12) dues à la modification d'un grand nombre de paramètres simultanés, elle parvient à décrocher le score minimal absolu du projet à l'époque 12 (**Val Loss : 0.1863**).

Comparaison brute

Le fine-tuning permet d'obtenir un modèle légèrement plus spécialisé et performant au prix d'une instabilité mineure durant les premières époques, tandis que le gel de l'encodeur reste la méthode la plus sûre, régulière et rapide (économie de calcul sur la rétropropagation de l'encodeur).

QUESTION 10 : Tableau comparatif final : comparez SimpleUNet vs TransferUNet (frozen) vs TransferUNet (fine-tuned) sur les critères : mIoU, mDice, nb paramètres, temps d'entraînement/epoch, temps d'inférence. Interprétez les résultats et concluez sur le choix optimal.

Nous allons encore run une case de code pour obtenir les cases de performances manquantes (mIoU et mDice sur le Test Set pour les deux modèles de Transfert Learning) et obtenir les temps exacts d'inférence.

```
===== SCORES COMPLÉMENTAIRES POUR LE TABLEAU (Q10) =====  
TRANSFER UNET (FROZEN)      | mIoU: 0.8126 | mDice: 0.8901 | Inférence:  
10.73 ms/image  
TRANSFER UNET (FINE-TUNED) | mIoU: 0.8138 | mDice: 0.8908 | Inférence:  
10.10 ms/image
```

Critère d'évaluation	SimpleUNet (From Scratch)	TransfertUNet (Encoder Frozen)	TransfertUNet(Fine Tuned)
mIoU(Test)	0.7765	0.8126	0.8138
mDice(Test)	0.8662	0.8901	0.8908
Nb. Paramètres	7 763 107 (tous actifs)	47 020 003 (23.5 M actifs)	47 020 003 (tous actifs)
Temps d'entraînement /epoch	~ 113.2 s	~77.1 s	~118.4 s
Temps d'inférence (moyen)	~4.80 ms/img	~10.73 ms/img	~10.10 ms / img

Tableau comparatif final

Interprétation des résultats :

1. **Saut de Performance (Qualité sémantique)** : Les deux configurations de **TransferUNet** surclassent nettement le **SimpleUNet**, avec un gain immédiat de **+3.7% sur le mIoU**. L'extraction de caractéristiques géométriques riches issues du pré-entraînement sur ImageNet (bords, textures complexes, motifs de pelage) fournit au décodeur des informations infiniment plus matures que des filtres initialisés de manière aléatoire.
2. **Frozen vs Fine-Tuned** : Le fine-tuning complet offre le meilleur score absolu (mIoU = 0.8138), mais le modèle à l'encodeur gelé (*Frozen*) se positionne juste derrière à un cheveu près (0.8126).
3. **Efficacité Computationnelle** : La version *Frozen* est la plus rapide à s'entraîner (**~77s par époque** au lieu de ~113s et ~118s). N'ayant pas à propager les gradients à travers les 23 millions de paramètres du squelette ResNet50, elle économise une quantité colossale de bande passante mémoire sur le GPU.
4. **Vitesse d'Inférence** : Le **SimpleUNet** reste le plus rapide lors de la prédiction en production (~4.8 ms) grâce à sa légèreté structurelle (7.7M de paramètres totaux contre 47M pour le ResNet50).

Conclusion sur le choix optimal :

Je trouve que le choix optimal dépend du cas d'usage :

- **En phase de Recherche / Contrainte de ressources** : La stratégie **TransferUNet (Frozen)** est le choix le plus pragmatique. Elle converge beaucoup plus vite tout en offrant des performances sémantiques quasiment identiques au Fine-Tuning complet.
- **Recherche de la performance pure** : Le **TransferUNet (Fine-Tuned)** reste l'architecture reine pour maximiser la finesse géométrique des masques au prix d'un calcul plus lourd.

Github : <https://github.com/Angeraiyane/CNN-FOR-IMAGES-SEGMENTATION>